

Lab 3 FEM 软体与布料仿真实现报告

1 软体模拟部分说明

软体模拟的核心代码位于 `src/VCX/Labs/3-FEM/FEMSystem.h/.cpp` 和 `CaseSoftBody.h/.cpp`。系统使用规则长方体网格生成四面体网格，默认尺寸为 $8\text{m} \times 2\text{m} \times 2\text{m}$ ，网格分辨率为 $8 \times 2 \times 2$ 。每个立方体单元被拆成 6 个四面体；左端 $i = 0$ 的顶点固定，其余顶点作为自由质点参与显式积分。

每个四面体在初始化时保存四个顶点索引、Rest Shape 矩阵逆 $\mathbf{D}_m^{-1}$ 和静止体积 V 。其中

$$\mathbf{D}_m = \begin{bmatrix} \mathbf{X}_1 - \mathbf{X}_0 & \mathbf{X}_2 - \mathbf{X}_0 & \mathbf{X}_3 - \mathbf{X}_0 \end{bmatrix}, \quad V = \frac{|\det(\mathbf{D}_m)|}{6}.$$

质量按体积和密度计算，再平均分配到四面体的四个顶点。固定点质量设为无穷大，并在每步积分时强制回到 Rest Position。

Listing 1: 四面体初始化时记录 Rest Shape 信息

```
1 void FEMSystem::AddTet(std::size_t i0, std::size_t i1,
2                       std::size_t i2, std::size_t i3) {
3     glm::mat3 Dm = makeColumnMat(
4         RestPositions[i1] - RestPositions[i0],
5         RestPositions[i2] - RestPositions[i0],
6         RestPositions[i3] - RestPositions[i0]);
7     float det = glm::determinant(Dm);
8     if (det < 0.f) {
9         std::swap(i1, i2);
10        Dm = makeColumnMat(
11            RestPositions[i1] - RestPositions[i0],
12            RestPositions[i2] - RestPositions[i0],
13            RestPositions[i3] - RestPositions[i0]);
14        det = glm::determinant(Dm);
15    }
16
17    Tets.push_back(Tet {
18        .Idx      = { i0, i1, i2, i3 },
19        .InvRest   = glm::inverse(Dm),
20        .RestVolume = std::abs(det) / 6.f,
21    });
22 }
```

每一小步仿真先清空力，再加入重力、速度阻尼和用户外力。对每个四面体，当前形状矩阵为

$$\mathbf{D}_s = \begin{bmatrix} \mathbf{x}_1 - \mathbf{x}_0 & \mathbf{x}_2 - \mathbf{x}_0 & \mathbf{x}_3 - \mathbf{x}_0 \end{bmatrix}, \quad \mathbf{F} = \mathbf{D}_s \mathbf{D}_m^{-1}.$$

基础模型使用 StVK 能量。记

$$\mathbf{G} = \frac{1}{2}(\mathbf{F}^T \mathbf{F} - \mathbf{I}), \quad \mathbf{S} = 2\mu \mathbf{G} + \lambda \text{tr}(\mathbf{G}) \mathbf{I},$$

则 First Piola–Kirchhoff stress 为

$$\mathbf{P} = \mathbf{F} \mathbf{S}.$$

离散力使用

$$\mathbf{H} = -V \mathbf{P} \mathbf{D}_m^{-T}.$$

$\mathbf{H}$ 的三列分别加到第 1、2、3 个顶点，第 0 个顶点受到负的列和，从而保证单元内部力守恒。

Listing 2: 由 deformation gradient 计算四面体内力

```
1 glm::mat3 const Ds = makeColumnMat(
2     Positions[i1] - Positions[i0],
3     Positions[i2] - Positions[i0],
4     Positions[i3] - Positions[i0]);
5 glm::mat3 const F = Ds * tet.InvRest;
6 glm::mat3 const P = firstPiola(F, Model, lambda, mu);
7 glm::mat3 const H = -tet.RestVolume * P * glm::transpose(tet.InvRest);
8
9 Forces[i1] += col(H, 0);
10 Forces[i2] += col(H, 1);
11 Forces[i3] += col(H, 2);
12 Forces[i0] -= col(H, 0) + col(H, 1) + col(H, 2);
```

时间积分采用显式欧拉。自由顶点根据 $\mathbf{a}_i = \mathbf{f}_i/m_i$ 更新速度和位置，并对速度做上限裁剪以避免显式积分在极端参数下数值爆炸。默认物理参数采用作业文档推荐值：杨氏模量 20000，泊松比 0.2，密度 400，重力 0.05，阻尼 1.2，时间步长 0.001。场景默认停止，需要点击 Start Simulation 才开始仿真。

Listing 3: 显式欧拉积分与速度上限

```
1 glm::vec3 const accel = Forces[i] / Masses[i];
2 Velocities[i] += accel * dt;
3 float const speed = glm::length(Velocities[i]);
4 if (speed > MaxVelocity) {
5     Velocities[i] *= MaxVelocity / speed;
6 }
7 Positions[i] += Velocities[i] * dt;
```

为了让仿真速度不直接依赖帧率，软体部分在渲染时使用 `Engine::GetDeltaTime()` 做时间累积。每帧按照固定小步长 `_timeStep` 消耗累积时间，同时通过 `Max Steps / Frame` 限制单帧最多子步数。

Listing 4: 软体模拟的固定时间步推进

```
1 float const frameDt = std::min(Engine::GetDeltaTime(), 0.05f);
2 _timeAccumulator += frameDt * _simulationSpeed;
```

```

3
4 int steps = 0;
5 int const maxSteps = std::max(1, _maxStepsPerFrame);
6 while (_timeAccumulator >= _timeStep && steps < maxSteps) {
7     _system.Step(_timeStep, _selectedParticle, frameControlForce);
8     _timeAccumulator -= _timeStep;
9     ++steps;
10 }

```

2 B1 说明：多种超弹性模型

Bonus 1 在同一个软件系统中加入材料模型选择，而不是同时显示多个软件。代码中定义了 `MaterialModel` 枚举,并在 UI 中通过 `Material Model` 下拉框切换 `StVK`、`Neo-Hookean` 和 `Corotated`。切换后同一个四面体网格会使用新的应力模型计算内力，因此可以在相同几何和交互条件下观察材料行为差异。

Listing 5: 材料模型枚举与 UI 选择

```

1 enum class MaterialModel : int {
2     StVK = 0,
3     NeoHookean = 1,
4     Corotated = 2,
5 };
6
7 char const * names[] = { "StVK", "Neo-Hookean", "Corotated" };
8 if (ImGui::Combo("Material Model", &_activeModel, names, 3)) {
9     _system.Model = static_cast<MaterialModel>(_activeModel);
10 }

```

三种模型共用同一套四面体离散和力汇总流程，区别只在 `firstPiola` 函数。`StVK` 模型使用上一节所述 $\mathbf{P} = \mathbf{F}\mathbf{S}$ 。`Neo-Hookean` 模型使用

$$\mathbf{P} = \mu(\mathbf{F} - \mathbf{F}^{-T}) + \lambda \log(J) \mathbf{F}^{-T}, \quad J = \det(\mathbf{F}).$$

实现中对 J 做了最小值裁剪，避免单元翻转或过度压缩时 $\log(J)$ 和逆矩阵产生非法值。

`Corotated` 模型先对 $\mathbf{F}$ 做极分解，取旋转部分 $\mathbf{R}$ ，再使用

$$\mathbf{P} = 2\mu(\mathbf{F} - \mathbf{R}) + \lambda(J - 1)J\mathbf{F}^{-T}.$$

极分解通过 Eigen 的 SVD 实现，并在 $\det(\mathbf{R}) < 0$ 时修正一列奇异向量，保证得到合法旋转矩阵。

Listing 6: 三种材料模型的 First Piola stress

```

1 glm::mat3 firstPiola(glm::mat3 const & F,
2                     MaterialModel model,
3                     float lambda,
4                     float mu) {
5     if (model == MaterialModel::NeoHookean) {

```

```

6         float const J = std::max(glm::determinant(F), 1e-4f);
7         glm::mat3 const FinvT = glm::transpose(glm::inverse(F));
8         return mu * (F - FinvT) + lambda * std::log(J) * FinvT;
9     }
10
11     if (model == MaterialModel::Corotated) {
12         glm::mat3 const R = polarRotation(F);
13         float const J = glm::determinant(F);
14         return 2.f * mu * (F - R)
15             + lambda * (J - 1.f) * J * glm::transpose(glm::inverse(F));
16     }
17
18     glm::mat3 const C = glm::transpose(F) * F;
19     glm::mat3 const G = 0.5f * (C - glm::mat3(1.f));
20     glm::mat3 const S = 2.f * mu * G
21         + lambda * traceMat(G) * glm::mat3(1.f);
22     return F * S;
23 }

```

3 B2 说明：布料模拟

Bonus 2 的布料代码位于 `ClothSystem.h/.cpp` 和 `CaseCloth.h/.cpp`。布料被建模为嵌入三维空间的二维三角形网格，默认分辨率为 24×24 ，尺寸为 $2\text{m} \times 2\text{m}$ 。网格初始位于 xz 平面上方，左侧两个角固定，其余顶点自由运动。

和三维软体不同，布料单个三角形的 Rest Shape 只需要二维坐标描述。初始化时为每个顶点保存三维位置 `RestPositions` 和二维参数坐标 `RestUV`。每个三角形保存

$$\mathbf{D}_m = \begin{bmatrix} \mathbf{u}_1 - \mathbf{u}_0 & \mathbf{u}_2 - \mathbf{u}_0 \end{bmatrix}, \quad A = \frac{|\det(\mathbf{D}_m)|}{2}.$$

质量根据面密度和三角形面积分配到三个顶点，默认参数使用文档建议值：杨氏模量 50，泊松比 0.3，面密度 0.5，重力 9.8，时间步长 0.001。为了让默认演示更稳定，速度阻尼设为 0.6，默认每帧只推进 1 个子步；需要更快运动时可以手动增加 `Steps / Frame`。

布料的当前形状矩阵为 3×2 ：

$$\mathbf{D}_s = \begin{bmatrix} \mathbf{x}_1 - \mathbf{x}_0 & \mathbf{x}_2 - \mathbf{x}_0 \end{bmatrix}, \quad \mathbf{F} = \mathbf{D}_s \mathbf{D}_m^{-1}.$$

随后在二维参数域中计算

$$\begin{aligned} \mathbf{C} &= \mathbf{F}^T \mathbf{F}, & \mathbf{G} &= \frac{1}{2}(\mathbf{C} - \mathbf{I}_2), \\ \mathbf{S} &= 2\mu \mathbf{G} + \lambda \text{tr}(\mathbf{G}) \mathbf{I}_2, & \mathbf{P} &= \mathbf{F} \mathbf{S}. \end{aligned}$$

离散力矩阵为

$$\mathbf{H} = -\mathbf{A} \mathbf{P} \mathbf{D}_m^{-T}.$$

$\mathbf{H}$ 是 3×2 矩阵，其两列分别给三角形第 1、2 个顶点，第 0 个顶点受到负列和。

Listing 7: 布料三角形的 3x2 FEM 内力

```

1 glm::mat2x3 const Ds(
2     Positions[i1] - Positions[i0],
3     Positions[i2] - Positions[i0]);
4 glm::mat2x3 const F = Ds * tri.InvRest;
5 glm::mat2 const C = glm::transpose(F) * F;
6 glm::mat2 const G = 0.5f * (C - glm::mat2(1.f));
7 glm::mat2 const S = 2.f * mu * G
8     + lambda * traceMat(G) * glm::mat2(1.f);
9 glm::mat2x3 const P = F * S;
10 glm::mat2x3 const H = -tri.RestArea * P
11     * glm::transpose(tri.InvRest);
12
13 glm::vec3 const f1 = mat2x3Col0(H);
14 glm::vec3 const f2 = mat2x3Col1(H);
15 Forces[i1] += f1;
16 Forces[i2] += f2;
17 Forces[i0] -= f1 + f2;

```

除了面内 FEM 弹性力，布料还加入了简单的弯曲弹簧。实现上对横向或纵向相隔两个网格点的顶点建立 BendEdge，保存静止长度。每步对这些边施加 Hooke 型长度恢复力，用于减小布料过度折叠和数值抖动。

Listing 8: 布料弯曲弹簧

```

1 glm::vec3 const x = Positions[edge.I1] - Positions[edge.I0];
2 float const len = glm::length(x);
3 if (len > 1e-7f) {
4     glm::vec3 const dir = x / len;
5     glm::vec3 const f = BendingStiffness
6         * (len - edge.RestLength) * dir;
7     if (!Fixed[edge.I0]) Forces[edge.I0] += f;
8     if (!Fixed[edge.I1]) Forces[edge.I1] -= f;
9 }

```

4 交互方式指南

- 构建运行：在仓库根目录执行 `xmake run lab3`。
- 软体材料：软体场景中通过 Material Model 下拉框切换 StVK、Neo-Hookean、Corotated。
- 选择控制点：Control Particle 滑条可以选择任意顶点；软体的 Select Free End 会自动选择自由端顶点，布料的 Select Free Corner 会自动选择自由角点。
- 键盘施力：画面获得焦点后，J/L 沿 $-X/+X$ 方向施力，U/O 沿 $+Y/-Y$ 方向施力，I/K 沿 $-Z/+Z$ 方向施力。

- 鼠标拖拽: 按住 **Z** + 鼠标左键并拖动画面, 可以在当前相机视平面内对选中顶点施加外力。拖拽强度由 **Mouse Force Scale** 控制。
- 按钮施力: UI 中的 **+X/-X/+Y/-Y/+Z/-Z** 按钮会对当前选中顶点施加一次方向力, 大小由 **Control Force** 控制。
- 可视化: 黄色大点表示当前选中的控制顶点; 红色点表示固定顶点; **Surface**、**Wireframe**、**Particles**、**Fixed Points** 可分别控制表面、线框、粒子和固定点显示。